

Fiche de révision — Architectures logicielles (option S7)

Méthodes clés pour traiter n'importe quel exercice d'architecture logicielle

1. Qu'est-ce qu'une architecture logicielle ?

L'architecture décrit l'organisation d'un système en composants, leurs responsabilités et leurs interactions, en vue de satisfaire des attributs de qualité (performance, sécurité, évolutivité...) — pas seulement de répondre aux fonctionnalités.

2. Styles architecturaux — tableau de référence

Style	Principe	Avantage clé	Limite
Monolithique en couches (layered)	couches empilées (présentation, métier, données), chaque couche n'appelle que la couche inférieure	simple à comprendre et développer	difficile à faire évoluer/scaler indépendamment
Client-serveur	un serveur centralise les ressources, des clients y accèdent	centralisation du contrôle et des données	point unique de défaillance, charge concentrée
Microservices	services indépendants, déployables séparément, communiquant via API/réseau	scalabilité et déploiement indépendants	complexité opérationnelle, latence réseau, cohérence distribuée
Pipe-and-filter	des filtres transforment des données transmises via des flux (pipes)	composants réutilisables et testables isolément	peu adapté aux interactions bidirectionnelles
Orientée événements (event-driven)	les composants réagissent à des événements publiés de façon asynchrone	faible couplage, bonne réactivité	flux de contrôle difficile à suivre, débogage complexe
Hexagonale (ports & adapters)	le cœur métier ne dépend d'aucune techno externe ; les entrées/sorties passent par des ports/adaptateurs	métier testable indépendamment de l'infrastructure	structure initiale plus lourde à mettre en place

3. MVC / MVVM

- **MVC** : Modèle (données+logique métier), Vue (affichage), Contrôleur (reçoit les entrées utilisateur, met à jour le modèle et choisit la vue).
- **MVVM** : la Vue se lie (binding) directement au ViewModel, qui expose l'état de présentation sans référencer la Vue → utile avec des frameworks à liaison de données automatique.
- Méthode d'exercice : toujours identifier qui dépend de qui — dans MVC le contrôleur dépend du modèle et de la vue, mais le modèle ne dépend jamais de la vue.

4. Attributs de qualité (quality attributes) — vocabulaire à mobiliser

Attribut	Définition courte	Tactique architecturale typique
Disponibilité (availability)	le système répond même en cas de panne partielle	redondance, basculement automatique (failover), health checks
Scalabilité	capacité à absorber une charge croissante	réplication horizontale, équilibrage de charge, cache
Maintenabilité	facilité à modifier/faire évoluer le système	faible couplage, forte cohésion, séparation des responsabilités
Sécurité	protection contre les accès/usages non autorisés	authentification, autorisation, chiffrement, moindre privilège
Performance	temps de réponse / débit du système	mise en cache, traitement asynchrone, optimisation des accès données

Méthode d'exercice « choisir/justifier une architecture » : partir des attributs de qualité exigés par l'énoncé (souvent en gras ou en priorité), éliminer les styles qui les compromettent, puis choisir celui qui répond au meilleur compromis — toujours argumenter en termes de compromis (trade-off), jamais de solution parfaite.

5. Documenter une architecture

- **Vues multiples** (souvent citées : logique, processus, déploiement, physique) : une seule vue ne suffit jamais à décrire complètement une architecture.
- **ADR (Architecture Decision Record)** : document court qui trace une décision d'architecture — contexte, décision prise, conséquences/compromis. Utile pour justifier un choix en examen : contexte → options considérées → décision → justification.

6. Communication entre composants

Mode	Exemple	Quand l'utiliser
Synchrone (requête/réponse)	appel REST/API	le client a besoin immédiatement du résultat
Asynchrone (message/événement)	file de messages, bus d'événements	découpler l'émetteur du récepteur, absorber les pics de charge

7. Pièges classiques en examen

- Répondre par un style architectural sans le relier explicitement aux attributs de qualité demandés par l'énoncé.
- Confondre architecture (structure globale, composants/qualité) et conception détaillée (design patterns au niveau des classes).
- Présenter les microservices comme toujours supérieurs : ils ont un coût opérationnel réel, à mentionner systématiquement en contrepartie.
- Oublier qu'une même architecture peut être vue sous plusieurs angles (logique vs déploiement) — préciser de quelle vue on parle.